

Phase 1 — Full Code Change Report

Vyapar AI Universal AI Employee Platform

Status: IMPLEMENTED | **Date:** June 2, 2026

Goal: HONS-specific single-tenant MVP → multi-tenant-ready Universal AI Employee foundation

Principle: HONS is test tenant only (COMPANY_ID=hons). Code is industry-agnostic.

1. Files Changed (8 files)

File	Lines Before → After	Summary
company_manager.py	46 → 235	Central business abstraction layer (rewritten)
business_settings.py	36 → 48	Hardcoded HONS removed; loads via company_manager
products.py	70 → 67	Static PRODUCTS removed; dynamic by company_id
ai_employee_engine.py	147 → 171	Tenant-aware COMPANY_ID routing
main.py	249 → 277	Startup profile validation; GET / tenant status
company_profiles.json	37 → 52	Enriched generic tenant schema
env.example	36 → 30	COMPANY_ID added; stale BUSINESS_* removed
README_DEPLOY.md	90 → 96	Tenant deployment docs

2. company_manager.py — REMOVED (entire old file)

REMOVED — 46 LINES

```
import json
import os

COMPANY_FILE = "company_profiles.json"

def load_company(company_id: str):
    if not os.path.exists(COMPANY_FILE):
        return None
    with open(COMPANY_FILE, "r", encoding="utf-8") as f:
        data = json.load(f)
    return data.get(company_id)

def get_company_summary(company_id: str) → str:
    company = load_company(company_id)
    if not company:
        return ""
    summary = f"""\
Company Name: {company.get("company_name")}
...
Products:
"""
    for product in company.get("products", []):
        summary += f"\n- {product['name']} = NPR {product['price']} ..."
    return summary.strip()
```

2b. company_manager.py — ADDED (new abstraction layer)

ADDED — KEY PUBLIC API (235 LINES TOTAL)

```
class CompanyProfileError(LookupError): ...

def get_active_company_id() → str:
    return os.getenv("COMPANY_ID", "hons").strip() or "hons"
```

```
def _load_all_profiles() → dict[str, Any]:
    # Phase 1: JSON file. Phase 2: swap for DB repository.
    ...

def get_company(company_id: str) → dict[str, Any] | None: ...
def load_company(company_id: str) → dict[str, Any] | None: # backward-compat alias
def require_company(company_id: str) → dict[str, Any]: ...
def get_company_industry(company_id: str) → str: ...
def get_company_contact(company_id: str) → dict[str, str]: ...
def get_company_policies(company_id: str) → dict[str, str]: ...
def get_company_rules(company_id: str) → list[str]: ...
def get_catalog_label(company_id: str) → str: ...
def get_company_products(company_id: str) → list[dict[str, Any]]: ...
def get_company_summary(company_id: str) → str: ...
```

3. business_settings.py — REMOVED

REMOVED — HARDCODED HONS CONSTANTS

```
BUSINESS_NAME = "Himalayan Online Service Pvt. Ltd."
BUSINESS_TYPE = "Internet Service Provider"
BUSINESS_LOCATION = "Head Kamal Pokhari, Kathmandu, Nepal"
BUSINESS_PHONE = "+977-1-4541063"
BUSINESS_TOLL_FREE = "16600149520"
BUSINESS_EMAIL = "info@hons.com.np"
SUPPORT_HOURS = "6:00 AM to 7:00 PM"
INSTALLATION_CHARGE = "Free"
ROUTER_ONU = "Free Router / ONU"

def business_context_to_prompt() → str:
    return f"""
Business Profile:
- Company Name: {BUSINESS_NAME}
...
Important Business Rules:
- Always identify the company as Himalayan Online Service Pvt. Ltd. when needed.
- Do not invent package prices.
...""".strip()
```

3b. business_settings.py — ADDED

ADDED — DYNAMIC TENANT PROMPT BUILDER

```
from company_manager import (
    get_company_contact, get_company_industry,
    get_company_policies, get_company_rules, require_company,
)

def business_context_to_prompt(company_id: str) → str:
    company = require_company(company_id)
    contact = get_company_contact(company_id)
    policies = get_company_policies(company_id)
    rules = get_company_rules(company_id)
    lines = [
        "Business Profile:",
        f"- Company ID: {company.get('company_id', company_id)}",
        f"- Company Name: {company.get('company_name') or company_id}",
        f"- Business Type: {company.get('business_type', 'N/A')}",
        f"- Industry: {get_company_industry(company_id)}",
        f"- Location: {company.get('location', 'N/A')}",
        # + contact, policies, rules (all from JSON)
    ]
    return "\n".join(lines).strip()
```

4. products.py — REMOVED

REMOVED — STATIC PRODUCTS LIST + HONS-SPECIFIC FALLBACK

```
PRODUCTS = [
    {"name": "100 Mbps Internet Package", "price": "NPR 10,000",
     "description": "...Himalayan Online Service...", "tags": [...]},
    # 150 Mbps, 200 Mbps ...
]

def search_products(query: str, top_n: int = 3):
    for product in PRODUCTS: # static iteration
        ...

def products_to_prompt(items):
    if not items:
        return ""
    Available internet packages:
    - 100 Mbps: NPR 10,000 for 13 months
    ... ""
```

4b. products.py — ADDED

ADDED — TENANT-SCOPED DYNAMIC CATALOG

```
from company_manager import (
    get_active_company_id, get_catalog_label,
    get_company_policies, get_company_products,
)

def search_products(query: str, top_n: int = 3, company_id: str | None = None):
    tenant_id = company_id or get_active_company_id()
    for product in get_company_products(tenant_id):
        # same scoring logic, dynamic source
        ...

def products_to_prompt(items, company_id: str | None = None):
    tenant_id = company_id or get_active_company_id()
    catalog = get_catalog_label(tenant_id) # generic label, not "internet packages"
    if not items:
        all_products = get_company_products(tenant_id)
        # fallback from JSON, not hardcoded Mbps lines
```

5. ai_employee_engine.py — REMOVED / CHANGED

REMOVED

```
# No company_manager import
# generate_employee_reply(user_id, text) — no company_id param

product_items = search_products(clean_text, top_n=3)
system_prompt = compose_system_prompt(
    business_block=business_context_to_prompt(), # no company_id
    product_block=products_to_prompt(product_items),
)
```

ADDED

```
from company_manager import CompanyProfileError, get_active_company_id, require_company

_UNCONFIGURED_REPLY = "This AI employee is not configured yet. Please contact support."

def _resolve_company_id(company_id: str | None = None) → str: ...
def _ensure_company_configured(company_id: str) → bool: ...

async def generate_employee_reply(user_id: str, text: str, company_id: str | None = None):
    tenant_id = _resolve_company_id(company_id)
    # memory + direct recall unchanged (works without tenant)

    if not _ensure_company_configured(tenant_id):
        save_chat_turn(user_id, "assistant", _UNCONFIGURED_REPLY)
        return _UNCONFIGURED_REPLY

    product_items = search_products(clean_text, top_n=3, company_id=tenant_id)
    system_prompt = compose_system_prompt(
        business_block=business_context_to_prompt(tenant_id),
        product_block=products_to_prompt(product_items, company_id=tenant_id),
    )
    logger.info("EMPLOYEE_REPLY_GENERATED user_id=%s company_id=%s", user_id, tenant_id)
```

6. main.py — REMOVED / ADDED

REMOVED FROM LIFESPAN

```
logger.info("APP_STARTUP mode=%s port=%s", TELEGRAM_MODE, PORT)

@app.get("/")
async def root():
    return {"service": APP_NAME, "ok": True, "mode": TELEGRAM_MODE}
```

ADDED TO LIFESPAN + ROOT

```
from company_manager import CompanyProfileError, get_active_company_id, require_company

company_id = get_active_company_id()
try:
    company = require_company(company_id)
    logger.info("COMPANY_PROFILE_LOADED company_id=%s industry=%s name=%s",
        company_id, company.get("industry"), company.get("company_name"))
except CompanyProfileError:
    logger.error("COMPANY_PROFILE_MISSING company_id=%s — AI replies will return configuration error", company_id)

logger.info("APP_STARTUP mode=%s port=%s company_id=%s", TELEGRAM_MODE, PORT, company_id)

@app.get("/")
async def root():
    company_id = get_active_company_id()
    try:
        require_company(company_id); company_loaded = True
    except CompanyProfileError:
        company_loaded = False
    return {"service": APP_NAME, "ok": True, "mode": TELEGRAM_MODE,
        "company_id": company_id, "company_loaded": company_loaded}
```

7. company_profiles.json — BEFORE → AFTER

BEFORE (MINIMAL, NO INDUSTRY/RULES/TAGS)

```
{
  "hons": {
    "company_name": "Himalayan Online Service Pvt. Ltd.",
    "products": [{ "name": "100 Mbps", "price": 10000, "duration_months": 13}],
    "installation_charge": "Free", "router": "Free", "onu": "Free"
  }
}
```

AFTER (GENERIC MULTI-INDUSTRY SCHEMA)

```
{
  "hons": {
    "company_id": "hons",
    "industry": "isp",
    "currency": "NPR",
    "catalog_label": "products and services",
    "policies": { "installation_charge": "Free", "router": "Free", "onu": "Free" },
    "rules": [ "Do not invent prices...", "..."],
    "products": [{
      "name": "100 Mbps Internet Package",
      "price": 10000, "duration_months": 13,
      "tags": [ "100mbps", "internet", "package"],
      "description": "100 Mbps connectivity package..."
    }]
  }
}
```

8. env.example — REMOVED / ADDED

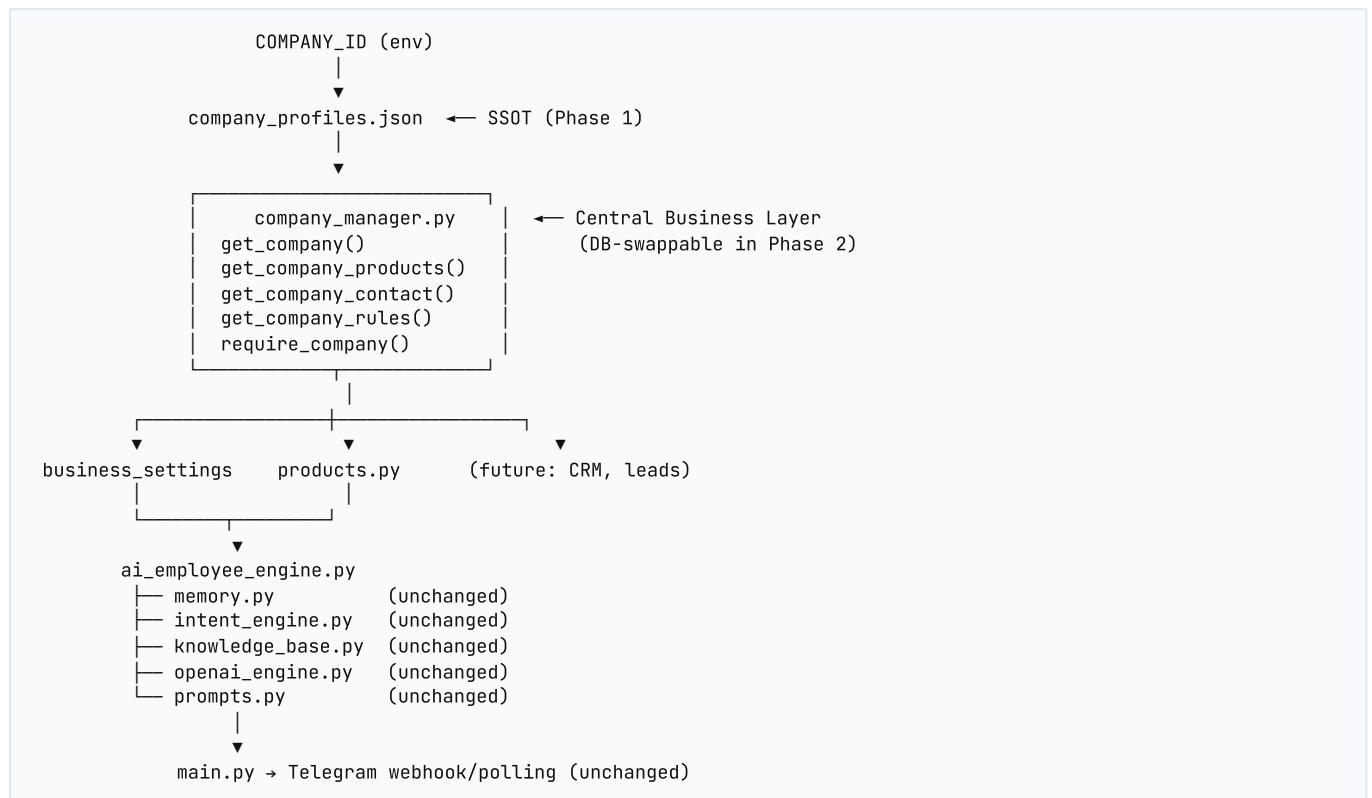
REMOVED (UNUSED VARS)

```
BUSINESS_NAME=Vyapar AI Employee
BUSINESS_TYPE=AI Employee Platform
SUPPORT_PHONE=+977-XXXXXXX
SUPPORT_EMAIL=support@example.com
SUPPORT_HOURS=Sun-Fri, 9 AM to 6 PM Nepal Time
ADMIN_USERNAME=Khemraj
```

ADDED

```
COMPANY_ID=hons
# COMPANY_PROFILES_FILE=/path/to/company_profiles.json
```

9. Architecture After Phase 1



10. Testing Checklist

#	Test	Command / Action	Expected
1	Profile loaded at startup	Start app with <code>COMPANY_ID=hons</code>	Log: <code>COMPANY_PROFILE_LOADED</code>
2	Root tenant status	GET /	<code>company_id=hons</code> , <code>company_loaded=true</code>
3	Invalid tenant	<code>COMPANY_ID=invalid</code> + message	"not configured yet" reply
4	Memory preserved	"Mero naam X ho" then "Mero naam ke ho?"	Direct recall works
5	Product search	"100 mbps price?" via Telegram	Product from JSON in prompt/reply
6	Telegram webhook	POST /telegram/webhook	Reply sent, no crash
7	Health check	GET /healthz	<code>{"ok":true,"database":"healthy"}</code>
8	Second tenant	Add hospital key to JSON, deploy with new <code>COMPANY_ID</code>	Different business context

11. Rollback Plan

1. Revert these 8 files to pre-Phase-1 versions (git checkout or backup restore)
2. Remove COMPANY_ID from Render env (old code ignored it)
3. No database migration was applied — memory tables unchanged
4. Restart service after rollback

12. Remaining Technical Debt

Item	Phase
JSON file SSOT — no hot reload, no admin CRUD	Phase 2 → DB
No company_id column on memory tables	Phase 2
Sync SQLAlchemy in async handlers	Phase 2
Blocking webhook (waits for OpenAI)	Phase 2
company_profile.json duplicate file (unused)	Cleanup
No automated tests	Phase 2
Keyword-only product/KB search	Phase 3
No lead engine / workflow / CRM	Phase 3

13. Recommended Phase 2

1. **DB tenant model** — companies, company_products, company_knowledge tables
2. **Repository swap** — replace _load_all_profiles() without changing public API
3. **company_id on memory** — tenant-scoped user data
4. **Alembic migrations**
5. **Admin API** — authenticated tenant CRUD (not full panel)
6. **Background worker** — async webhook processing
7. **Second test tenant** — hospital_demo industry seed in JSON/DB

14. Unchanged Files (Preserved)

memory.py · memory_db.py · memory_extractor.py · openai_engine.py · intent_engine.py · knowledge_base.py · prompts.py · Telegram webhook/polling logic in main.py